

IBMEC

Disciplina: Arquitetura de Computadores

NOTA DE AULA
Barramento e Processadores

Prof. Clayton J. A. Silva, MSc
clayton.silva@ibmec.edu.br

Versão 4 – 2026

RESUMO

Esta nota de aula aborda os dois elementos centrais de um sistema computacional moderno: o barramento e o processador. Na primeira parte (Seção 2, p. 4), são apresentados o conceito de barramento e suas funcionalidades, os modos de comunicação paralela e serial e as implicações da largura do barramento de endereço e de dados. Em seguida, trata-se da multiplexação de barramento (p. 6), com exemplo de leitura multiplexada. A hierarquia de barramentos em três níveis — com analogia ao tráfego urbano — e a taxa de erros em sistemas computacionais são apresentadas na p. 7. O controlador de barramento é detalhado em suas funções de arbitragem, temporização, detecção de erros e conversão de protocolo (p. 10). Os protocolos de barramento são apresentados em suas cinco dimensões, com destaque para o PCI Express como estudo de caso (p. 11). As duas formas de arbitragem centralizada — concessão não propagada e propagada ao longo da cadeia — e a arbitragem descentralizada são analisadas com diagramas de temporização (p. 15). As temporizações síncrona e assíncrona (handshake) encerram a parte de barramentos (p. 18).

A segunda parte (Seção 3, p. 20) parte do modelo de von Neumann e evolui até o processador multicore moderno. O pipeline de instruções é apresentado em seus seis estágios, com análise de desempenho, fórmulas e exemplo numérico (p. 21). O impacto de instruções de desvio no pipeline é discutido com diagrama de tempo comparativo (p. 23). Os processadores multicore têm sua organização interna, mecanismos de coerência de cache e Lei de Amdahl discutidos (p. 24). A seção final aborda o set de instruções, comparando RISC e CISC, descrevendo modos de endereçamento e o ciclo de instrução (p. 26).

1. INTRODUÇÃO: ELEMENTOS DO SISTEMA DE COMPUTAÇÃO

Um sistema computacional moderno é constituído por quatro grandes grupos de elementos que colaboram de forma integrada: a memória, o barramento, os processadores e os dispositivos de entrada e saída. Cada grupo exerce funções distintas, porém profundamente interdependentes — o desempenho global do sistema resulta da qualidade da interação entre todos eles. Esta nota de aula apresenta os conceitos fundamentais relativos ao barramento e ao processador, com ênfase nas técnicas que conferem eficiência a essas estruturas.

As referências principais são Tanenbaum e Austin (2013) e Stallings (2017). O conteúdo é complementado por fontes técnicas atualizadas — necessárias porque vários protocolos abordados nas referências clássicas tornaram-se obsoletos — e pelo conhecimento prático do docente.

2. BARRAMENTO

2.1 Conceito e funcionalidades

Antes de definir o barramento, é necessário compreender o que se entende por funcionalidade no contexto de sistemas digitais. Funcionalidade refere-se ao conjunto de serviços que um elemento do sistema oferece aos demais — aquilo que ele faz, não como o faz. No caso do barramento, as funcionalidades incluem a transferência de dados entre componentes, o endereçamento de destinos, a sinalização de controle e a arbitragem de acesso ao meio compartilhado.

O barramento é definido como um conjunto compartilhado de linhas de comunicação utilizado para interligar os componentes de um sistema computacional. Por meio dos barramentos, processador, memória e dispositivos de entrada e saída trocam informações e sinais de controle. O uso de barramentos reduz a quantidade de conexões físicas necessárias: uma mesma via pode ser aproveitada por múltiplos componentes, desde que existam mecanismos de coordenação de acesso.

A Figura 1, reproduzida de Stallings (2017), ilustra o esquema clássico de interconexão por barramento. A CPU, os módulos de memória e os dispositivos de E/S estão todos conectados a um mesmo conjunto de linhas, dividido funcionalmente em três grupos:

- Linhas de dados: transportam os valores que circulam entre os componentes.
- Linhas de endereço: indicam a origem ou o destino da informação.
- Linhas de controle: carregam sinais de coordenação — leitura/escrita de memória e E/S, confirmação de transferência (ACK), solicitação (*bus request*) e concessão de barramento (*bus grant*), clock, reset e interrupções.

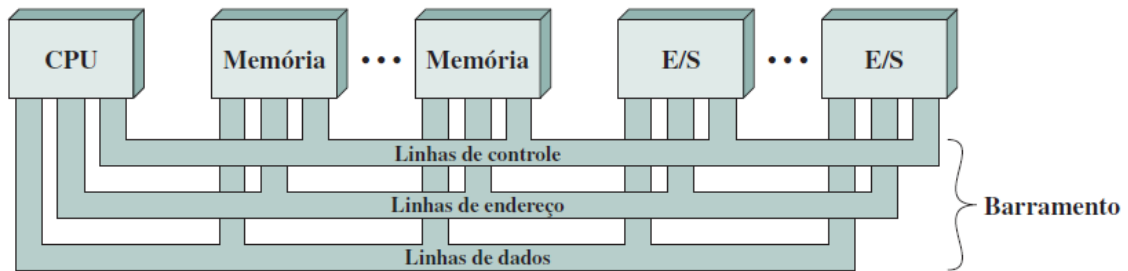


Figura 1 – Esquema de interconexão de barramento (Stallings, 2017, Fig. 3.16)

A Figura 1 evidencia a funcionalidade central do barramento: um único meio físico compartilhado que serve simultaneamente a todos os componentes do sistema, reduzindo de forma drástica o número de conexões necessárias. Sem o barramento, em um sistema com n componentes, seriam necessárias $n(n-1)/2$ conexões independentes.

2.2 Comunicação paralela e serial — implicações na largura do barramento

Há dois modos fundamentais de comunicação binária entre dispositivos, que diferem na forma como os bits são transmitidos ao longo do tempo.

Na comunicação **paralela**, múltiplos bits são transmitidos simultaneamente por linhas físicas distintas. Se o barramento tem w linhas de dados, transfere-se w bits a cada ciclo de clock. A Figura 2 ilustra um barramento paralelo de 8 linhas: todos os 8 bits do byte chegam ao receptor no mesmo instante de tempo.

Na comunicação **serial**, apenas uma linha física é usada e os bits são transmitidos um a um em instantes consecutivos. Para enviar 8 bits, são necessários 8 intervalos de tempo. O receptor reconstrói o byte à medida que os bits chegam em sequência.

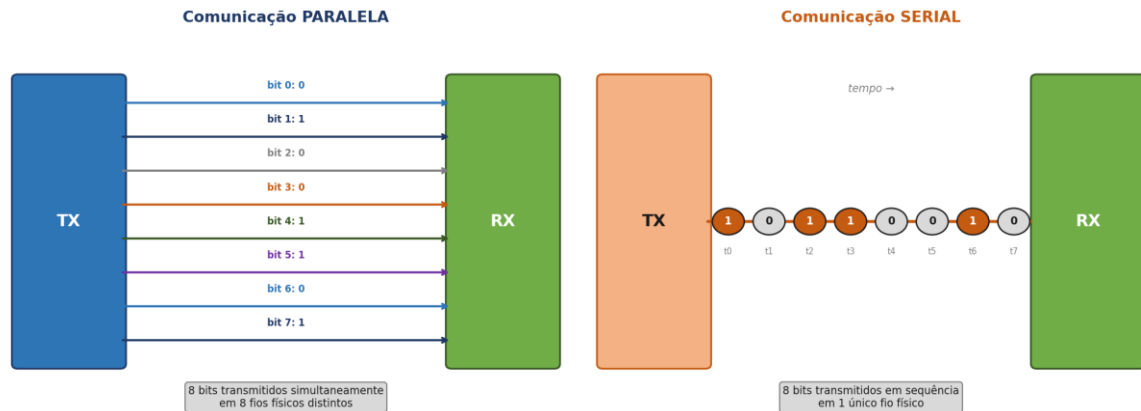


Figura 2 – Comunicação paralela (transmissão simultânea em múltiplos fios) × comunicação serial (transmissão sequencial em um único fio)

Cada modo possui vantagens e desvantagens. O barramento paralelo oferece maior *throughput* por ciclo, mas exige mais fios e apresenta o problema de *skew* — desalinhamento temporal entre bits. O barramento serial exige menos fios e é mais robusto a altas frequências, razão pela qual protocolos modernos — PCIe, USB, SATA, NVMe — adotam comunicação serial com múltiplas *lanes*.

2.2.1 Largura do barramento de endereço

O barramento de endereço tem n linhas e endereça diretamente 2^n posições de memória. CPUs de 16 bits podiam endereçar 64 KB; CPUs de 32 bits atingiam 4 GB; CPUs de 64 bits modernos endereçam teoricamente 16 EB (exabytes).

2.2.2 Largura do barramento de dados

O barramento de dados tem w linhas e transfere w bits por ciclo. Aumentar w eleva o *throughput* sem necessariamente aumentar a frequência. Barramentos de 8 bits transferem 1 byte/ciclo; de 64 bits (padrão DDR5), 8 bytes/ciclo.

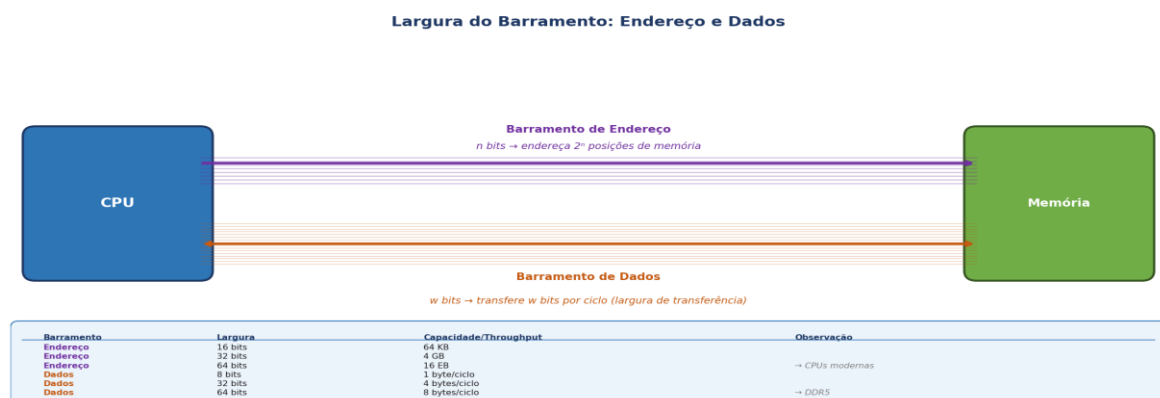


Figura 3 – Largura do barramento de endereço e de dados — capacidade endereçável e throughput por largura

2.3 Multiplexação de barramento

Uma estratégia para reduzir o número de linhas físicas sem sacrificar a capacidade de endereçamento é a multiplexação de barramento. A viabilidade dessa técnica decorre de uma assimetria temporal fundamental: o endereço precisa ser colocado no barramento antes dos dados — o componente destino precisa saber para onde os dados devem ir antes de recebê-los. Essa precedência natural torna possível usar as mesmas linhas físicas para transportar primeiro o endereço e depois os dados, em fases distintas de uma mesma transação.

No barramento multiplexado, o sinal *ALE* (*Address Latch Enable*) informa ao receptor o significado atual das linhas compartilhadas: ALE = 1 indica endereço; ALE = 0 indica dados.

A Figura 4 apresenta o diagrama de temporização da multiplexação. Na Fase 1 (t_0 a t_1), as linhas D7–D0 transportam o endereço com ALE em nível alto. No intervalo t_2 – t_3 ocorre o turnaround — as linhas mudam de função — e, a partir de t_3 , com ALE em nível baixo, os dados são transmitidos.

>> Exemplo — leitura multiplexada de 1 byte no endereço 0x1A00

Considere um microcontrolador de 8 bits com barramento multiplexado de 8 linhas (D7–D0):

Ciclo 1 (Fase Endereço): ALE = 1; D7–D0 = 0x1A (byte alto do endereço 0x1A00).

Ciclo 2 (Fase Endereço): ALE = 1; D7–D0 = 0x00 (byte baixo do endereço).

Turnaround: ALE vai a 0; as linhas ficam em alta impedância por ~1 ciclo.

Ciclo 3 (Fase Dados): ALE = 0, RD = 0 (ativo); D7–D0 = 0x42 (dado retornado pela memória).

Resultado: com 8 fios físicos, transmitiu-se um endereço de 16 bits e um dado de 8 bits — equivalente funcional de 24 fios em barramento não multiplexado.

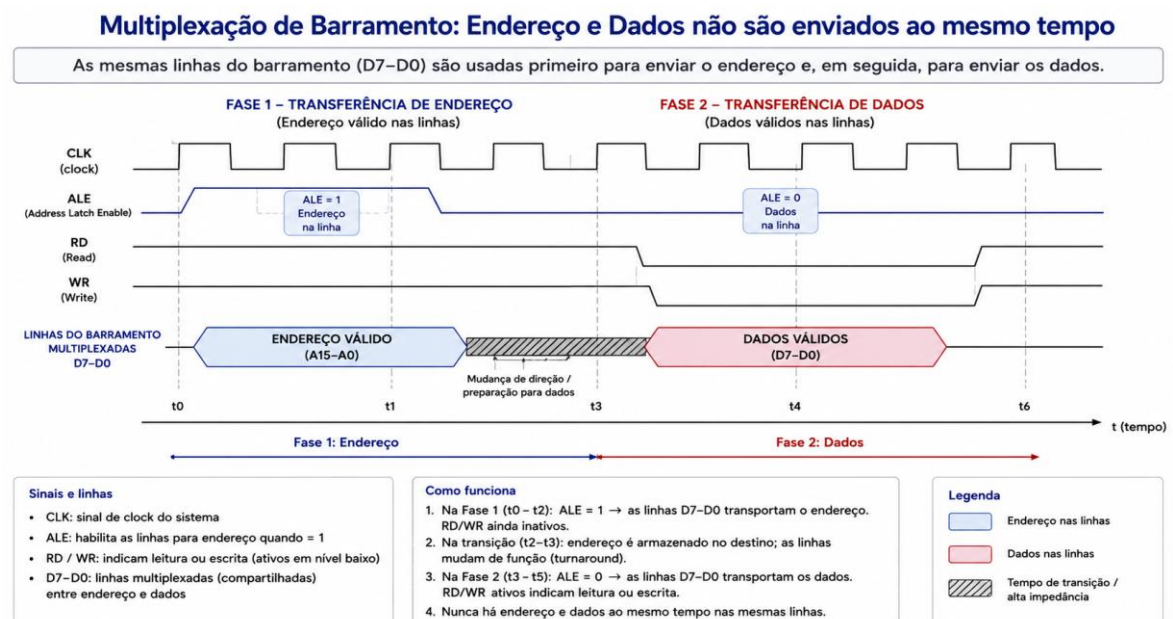


Figura 4 – Diagrama de temporização da multiplexação de barramento endereço/dados

A multiplexação reduz pinos e fios, sendo especialmente valiosa em microcontroladores. O custo é a latência do *turnaround* a cada transação.

2.4 Hierarquia de barramentos

Imagine o trânsito em uma grande cidade. Há autoestradas expressas projetadas para tráfego intenso e alta velocidade, avenidas de capacidade intermediária e ruas locais de baixo volume. Se um veículo lento precisasse usar a autoestrada expressa para chegar a uma rua local, bloquearia o tráfego de todo o sistema. A solução é criar vias específicas para cada tipo de fluxo, com pontos de conexão entre os níveis — exatamente o que a hierarquia de barramentos faz em um sistema computacional.

À medida que o número de dispositivos cresce, torna-se inviável utilizar um único barramento para todos os componentes: dispositivos distintos têm requisitos completamente diferentes de velocidade, largura de banda e custo. Conectar um disco rígido e a memória RAM ao mesmo barramento obrigaria a operar ambos na velocidade do dispositivo mais lento — uma perda inaceitável de desempenho.

Ademais, em qualquer meio compartilhado, múltiplas transmissões simultâneas geram colisões. Colisões produzem erros de comunicação. A taxa de erros em barramentos deve ser mantida extremamente baixa: sistemas computacionais modernos operam com taxas de erro de bit (*BER — Bit Error Rate*) da ordem de 10^{-12} a 10^{-15} — menos de um bit errado em um trilhão a um quadrilhão de bits transmitidos. O controlador de barramento garante essa baixíssima taxa por meio de arbitragem, temporização e detecção de erros.

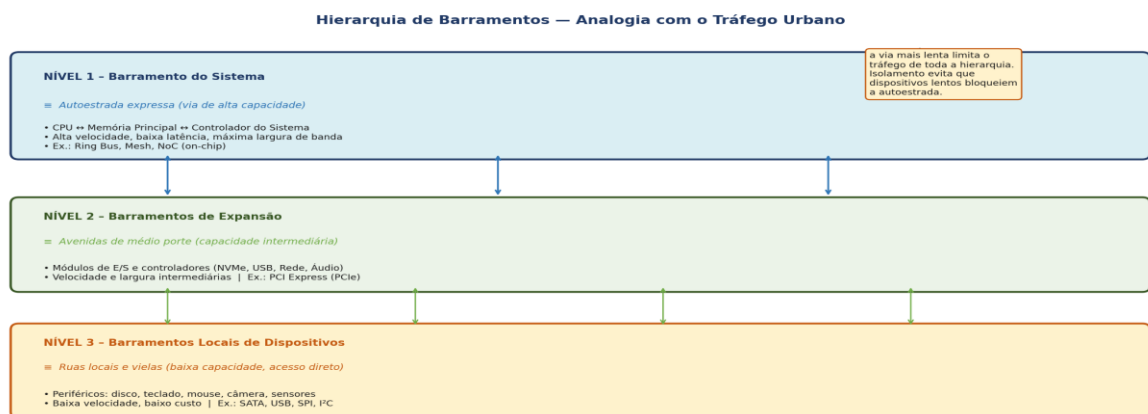


Figura 5 – Hierarquia de barramentos — analogia com o tráfego urbano

2.4.1 Nível 1 — Barramento do Sistema

O Nível 1 é o barramento de maior desempenho: interconecta a CPU, a memória principal e o controlador do sistema. Corresponde à autoestrada expressa. Em processadores modernos, essa interconexão é feita no *chip* por estruturas como *Ring Bus*, *Mesh* ou *Network-on-Chip* (NoC). A Figura 6 ilustra esse nível.

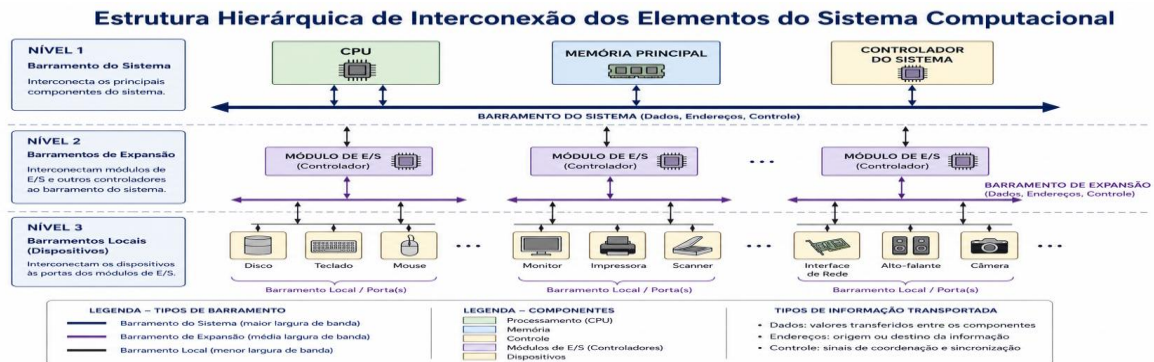


Figura 6 – Hierarquia de barramentos — Nível 1: Barramento do Sistema

2.4.2 Nível 2 — Barramentos de Expansão

O Nível 2 é formado pelos Barramentos de Expansão, que interligam os módulos de E/S ao barramento do sistema. Corresponde às avenidas de médio porte. O *PCIe Root Complex* é o árbitro típico desse nível nos sistemas atuais. A Figura 7 acrescenta o Nível 2.

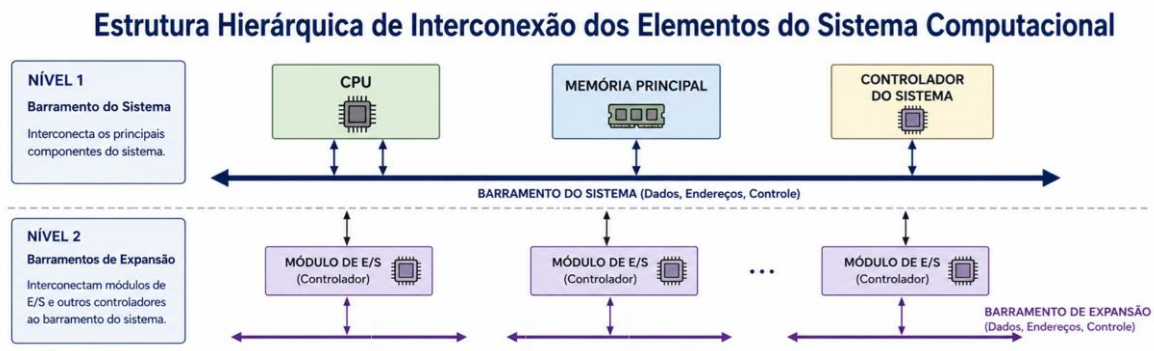


Figura 7 – Hierarquia de barramentos — Níveis 1 e 2: acréscimo dos Barramentos de Expansão

2.4.3 Nível 3 — Barramentos Locais de Dispositivos

O Nível 3 conecta os periféricos físicos às portas dos controladores do Nível 2. Corresponde às ruas locais e vielas: baixo custo, simplicidade, menor velocidade. Exemplos: SATA, USB, SPI, I²C, SMBus. A Figura 8 apresenta a hierarquia completa.

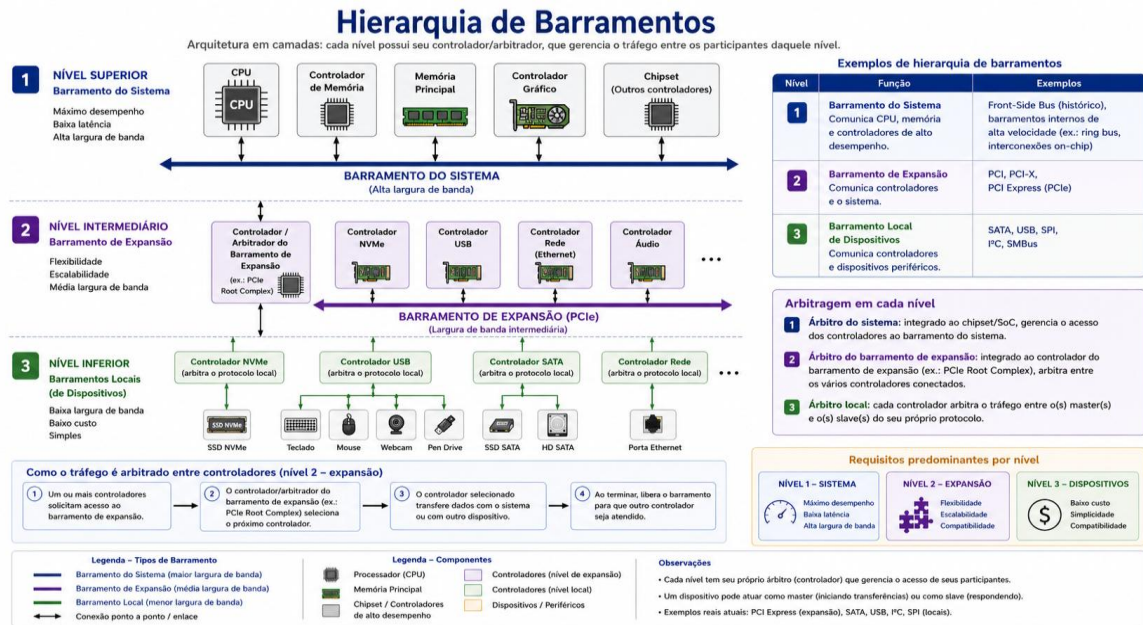


Figura 8 – Hierarquia de barramentos completa — três níveis com protocolos, requisitos e arbitragem por nível

O princípio fundamental da hierarquia é o isolamento: dispositivos lentos ficam confinados aos níveis inferiores e não bloqueiam o tráfego dos níveis superiores — tal como, na analogia do trânsito, um ônibus lento nas ruas locais não interfere na autoestrada expressa.

2.5 Controlador de barramento

2.5.1 Funções gerais

O controlador de barramento gerencia o uso do barramento compartilhado, garantindo que as comunicações ocorram de forma ordenada, sem colisões e com baixa taxa de erros. Suas funções principais são:

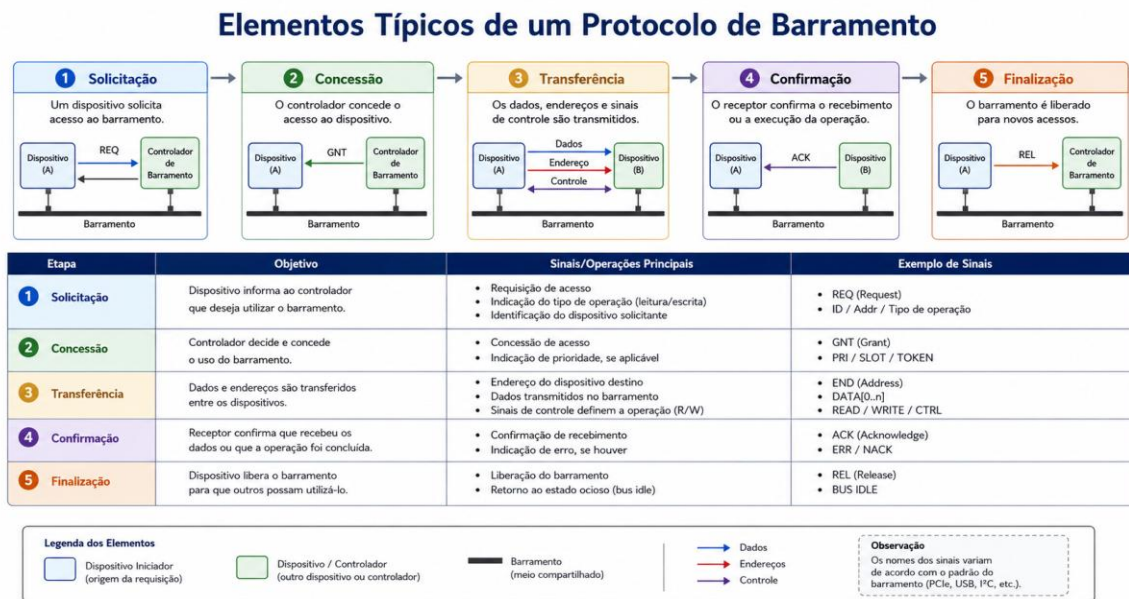
- **Controle de acesso (arbitragem):** em um barramento compartilhado, múltiplos dispositivos podem querer transmitir ao mesmo tempo. O controlador determina, segundo uma política de prioridade ou escalonamento, qual dispositivo tem acesso ao barramento em cada instante. Sem arbitragem, duas transmissões simultâneas colidem e corrompem os dados.
- **Geração e interpretação de sinais:** o controlador produz os sinais de controle necessários para cada fase da transação — sinais de leitura (RD) e escrita (WR), sinais de concessão (GNT) e confirmação (ACK) —, e interpreta os sinais gerados pelos *masters* e *slaves*.

- **Temporização (*timing*):** coordena a sequência e a duração dos sinais no barramento. Em barramentos síncronos, distribui o clock de referência; em assíncronos, gerencia a troca de sinais de *handshake*.
- **Deteção e sinalização de erros:** monitora o barramento em busca de condições anormais — violações de temporização, sinais fora de faixa, ausência de confirmação (*timeout*). Sinaliza ao dispositivo originador (NACK ou ERR) para que a transação seja repetida.
- **Conversão de protocolo (pontes):** quando dois barramentos de diferentes características precisam se comunicar, o controlador atua como ponte, convertendo sinais, frequências e protocolos entre os dois domínios.

2.5.2 Protocolos de barramento — conceito e dimensões

O protocolo de barramento é o conjunto completo de regras que define como os dispositivos devem se comunicar. Ele abrange cinco dimensões:

- **Lógica (protocolo):** define o formato das mensagens, os tipos de transação (leitura, escrita, confirmação), o esquema de endereçamento e os mecanismos de controle de fluxo.
- **Elétrica:** especifica os níveis de tensão dos sinais, a codificação (ex.: NRZ, 8b/10b, 128b/130b) e os requisitos de temporização — *setup time*, *hold time*, *slew rate*.
- **Mecânica:** define o formato físico dos conectores. Conectores incompatíveis mecanicamente impedem a conexão, mesmo que o protocolo lógico seja idêntico.
- **Funcional:** especifica os serviços oferecidos pelo protocolo — descoberta automática de dispositivos (*plug and play*), gerenciamento de energia, qualidade de serviço (QoS) e suporte a *hot-plug*.
- **Conformidade:** define os testes elétricos e de protocolo que um dispositivo deve passar para ser certificado como compatível.

 → Dados
 → Endereços
 → Controle

Observação

Os nomes dos sinais variam de acordo com o padrão do barramento (PCIe, USB, I²C, etc.).

Figura 9 – Dimensões de um padrão de barramento — exemplo PCI Express

2.5.3 Protocolos modernos em uso

Os livros-texto clássicos de arquitetura de computadores abordam protocolos como ISA, EISA e PCI convencional, hoje obsoletos. Os protocolos relevantes na prática atual são os seguintes.

DDR5 (*Double Data Rate 5*): barramento paralelo sincronizado entre CPU e RAM, com largura de 64 bits por canal, taxas de até 6,4 GT/s por pino e baixo consumo.

PCI Express (PCIe): protocolo serial *full-duplex* ponto a ponto. O PCIe 5.0 oferece 32 GT/s por *lane*; configurações x16 atingem 64 GB/s de *throughput* bidirecional.

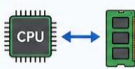
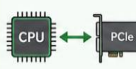
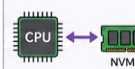
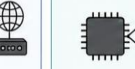
NVMe (*Non-Volatile Memory Express*): protocolo de armazenamento sobre PCIe, otimizado para SSDs de estado sólido.

USB4: baseado no protocolo *Thunderbolt 3*. Combina dados (até 40 Gbps), vídeo e alimentação elétrica em um único cabo.

Ethernet: protocolo serial para redes locais, com taxas de 1G a 100G+. Padronizado pelo IEEE 802.3.

I²C e SPI: protocolos seriais de baixo custo para sistemas embarcados e IoT, com 2 ou 4 fios.

Principais Protocolos de Barramento em Uso (2026)

	DDR5 (Memória)	PCI Express (Expansão Interna)	NVMe (Armazenamento)	USB4 (Periféricos Externos)	Ethernet (Redes)	I²C / SPI (Embarcados)
	 CPU ↔ Memória RAM	 CPU ↔ Dispositivos	 CPU ↔ SSD NVMe	 Computador ↔ Periféricos	 Dispositivo ↔ Rede	 MCU ↔ Sensores/Chips
 Função	Acesso à memória principal	Conexão com placas de expansão internas (GPU, rede, etc.)	Acesso a armazenamento de alta performance	Conexão com periféricos externos (áudio, vídeo, dados)	Comunicação em redes locais e na Internet	Comunicação entre circuitos integrados em sistemas embarcados
 Características	Alta largura de banda, sincronizado, baixo consumo	Muito alta velocidade, baixa latência, ponto a ponto	Protocolo otimizado para SSDs, alta taxa de transferência	Alta velocidade, alimentação elétrica e dados no mesmo cabo	Confiável, escalável, alta taxa de dados (1G, 2.5G, 10G, 25G, 40G, 100G+)	Baixo custo, simples, baixa velocidade, baixo consumo
 Onde é usado	Todos os computadores e servidores modernos	GPUs, SSDs NVMe, placas de rede, controladoras, etc.	SSDs NVMe em desktops, notebooks e servidores	Teclados, mouses, displays, docks, unidades externas, etc.	Computadores, servidores, switches, roteadores, etc.	Sensores, displays, memórias, PMICs, IoT, automação, etc.
 Tipo	Paralelo (sincronizado)	Serial (ponto a ponto)	Serial (sobre PCIe)	Serial (alta velocidade)	Serial (ponto a ponto)	Serial (2 ou 4 fios)

 **Resumo:** Esses protocolos formam a base das comunicações nos computadores modernos, desde a memória principal até os periféricos e a rede.

Figura 10 – Principais protocolos de barramento em uso (2026) — função, características e aplicações

2.5.4 O PCI Express como estudo de caso

O PCIe ilustra concretamente a complexidade de um protocolo moderno. Sua arquitetura é em camadas:

- Camada de Transação (*Transaction Layer*): gera e consome pacotes TLPs (*Transaction Layer Packets*) e implementa controle de fluxo por créditos.
- Camada de Enlace (*Data Link Layer*): gerencia a confiabilidade: detecta e corrige erros nos TLPs usando CRC de 32 bits e solicita retransmissão (ACK/NAK).
- Camada Física (*Physical Layer*): transmite os bits com sinalização diferencial, codificação 128b/130b e operação *full-duplex* — cada *lane* possui par TX e par RX independentes.

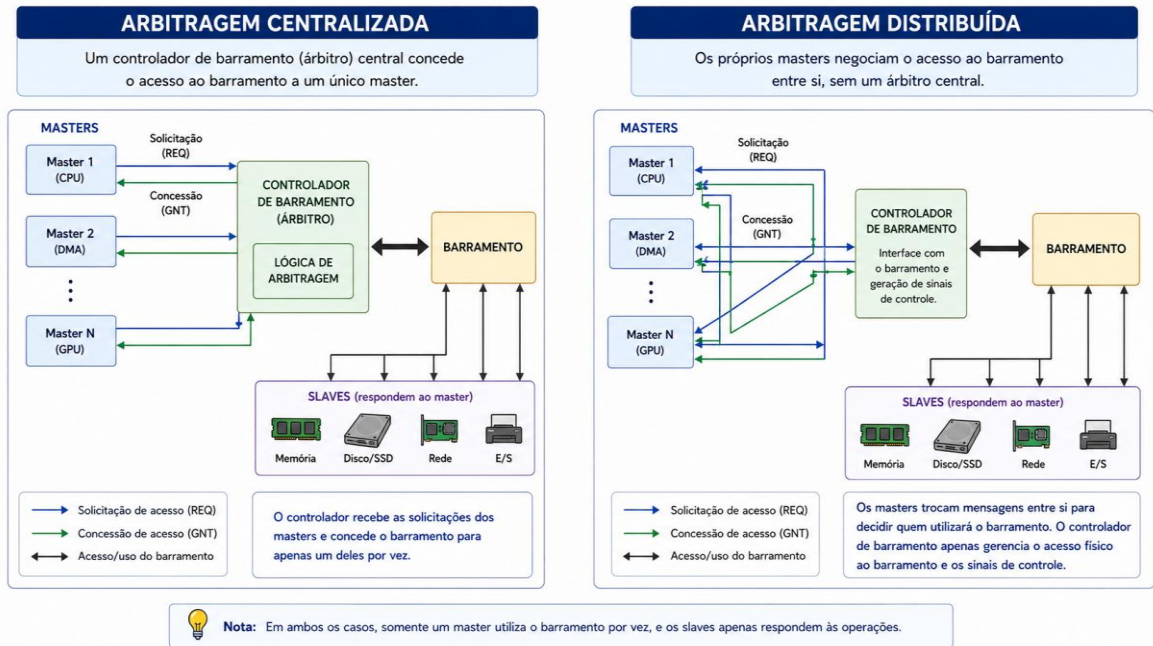


Figura 11 – PCI Express — visão geral: topologia, camadas de protocolo e formato de pacote (TLP)

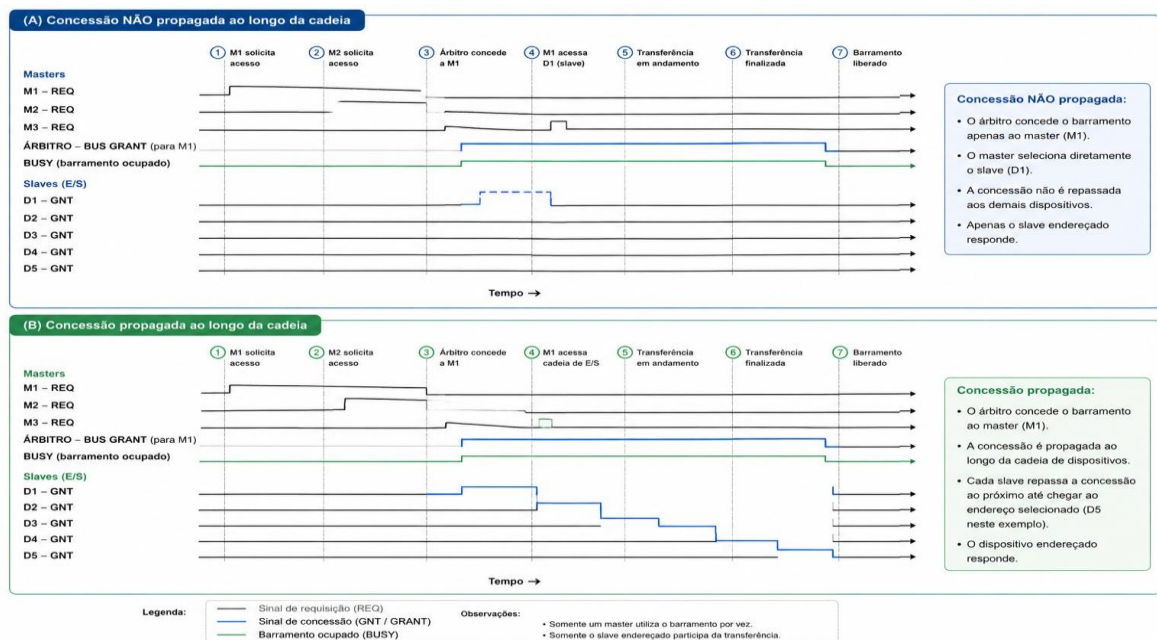


Figura 12 – Resumo das características do PCI Express por dimensão do padrão

2.5.5 Elementos típicos de uma transação de barramento

Independentemente do protocolo, toda transação segue cinco etapas: Solicitação (REQ), Concessão (GNT), Transferência (END + DATA + CTRL), Confirmação (ACK ou NACK) e Finalização (REL). As Figuras 13 a 16 apresentam essas etapas progressivamente.



Figura 13 – Transação — Etapa 1: Solicitação (REQ)

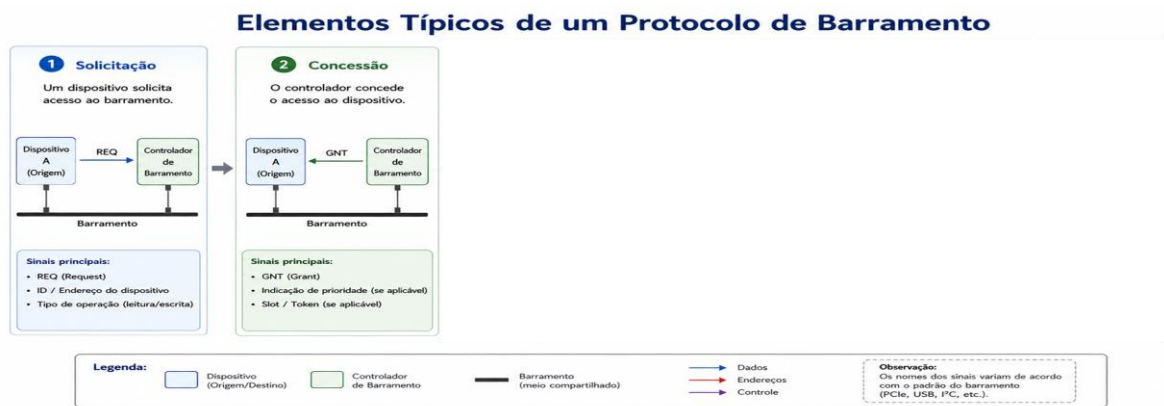


Figura 14 – Transação — Etapa 2: Concessão (GNT)

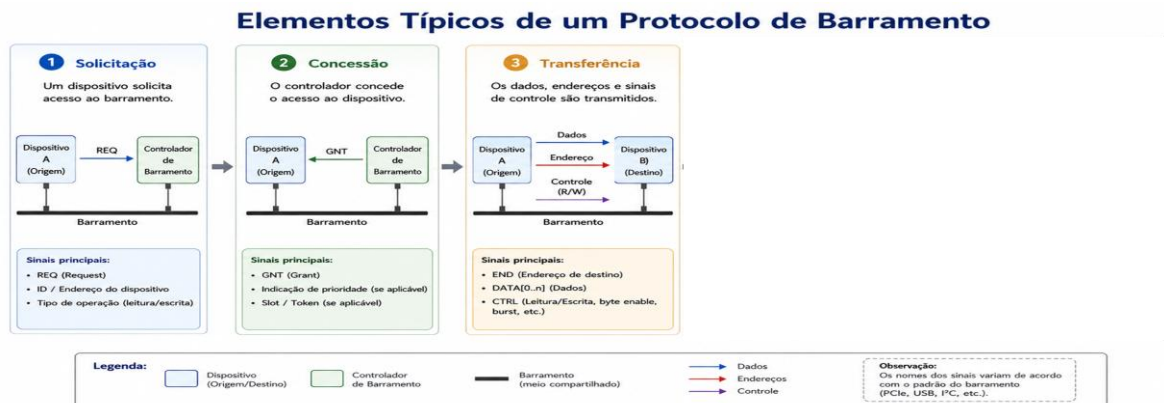
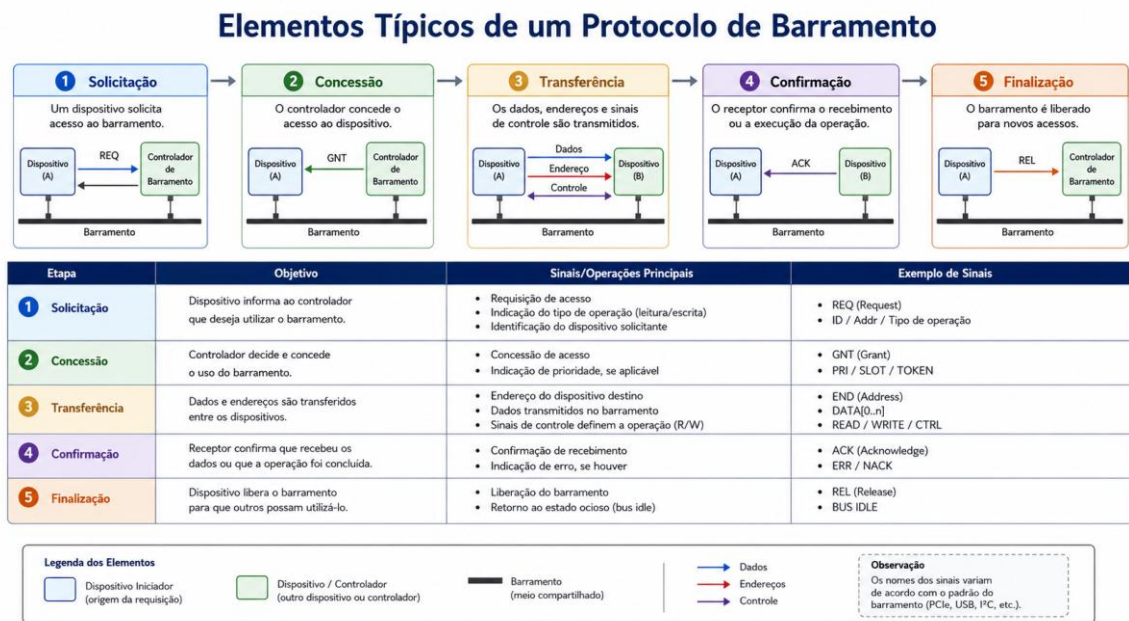


Figura 15 – Transação — Etapa 3: Transferência de dados, endereço e controle

 → Dados
 → Endereços
 → Controle

Observação

Os nomes dos sinais variam de acordo com o padrão do barramento (PCIe, USB, IFC, etc.).

Figura 16 – Transação completa — cinco etapas com sinais e objetivos

2.5.6 Arbitragem: *masters*, *slaves* e formas de arbitragem centralizada

Masters são dispositivos ativos que iniciam transferências; *slaves* são dispositivos passivos que apenas respondem. Quando múltiplos *masters* concorrem ao barramento, o árbitro decide quem obtém acesso. Duas abordagens principais existem: centralizada e descentralizada (distribuída).

Na arbitragem centralizada, existe um único árbitro que recebe todos os pedidos e emite as permissões. Dentro dessa abordagem, existem duas variantes quanto à forma de concessão do acesso ao slave:

Concessão não propagada: o árbitro concede o barramento diretamente ao *master* solicitante, que então acessa o *slave* destino diretamente, sem repassar a concessão. Apenas o *slave* endereçado participa da transação. É o modelo mais comum em sistemas modernos: CPU solicita barramento → árbitro concede → CPU acessa diretamente a memória ou o controlador desejado. Vantagem: latência baixa e previsibilidade.

Concessão propagada ao longo da cadeia (*daisy chain*): o árbitro concede o barramento ao *master*, mas o sinal GNT é propagado ao longo de uma cadeia de *slaves* — cada *slave* repassa o sinal ao próximo até que o *slave* endereçado o retém e responde. O *master* não precisa conhecer a posição física do *slave* na cadeia. Desvantagem: latência adicional proporcional à posição do *slave* na cadeia; um *slave* defeituoso pode interromper a propagação.

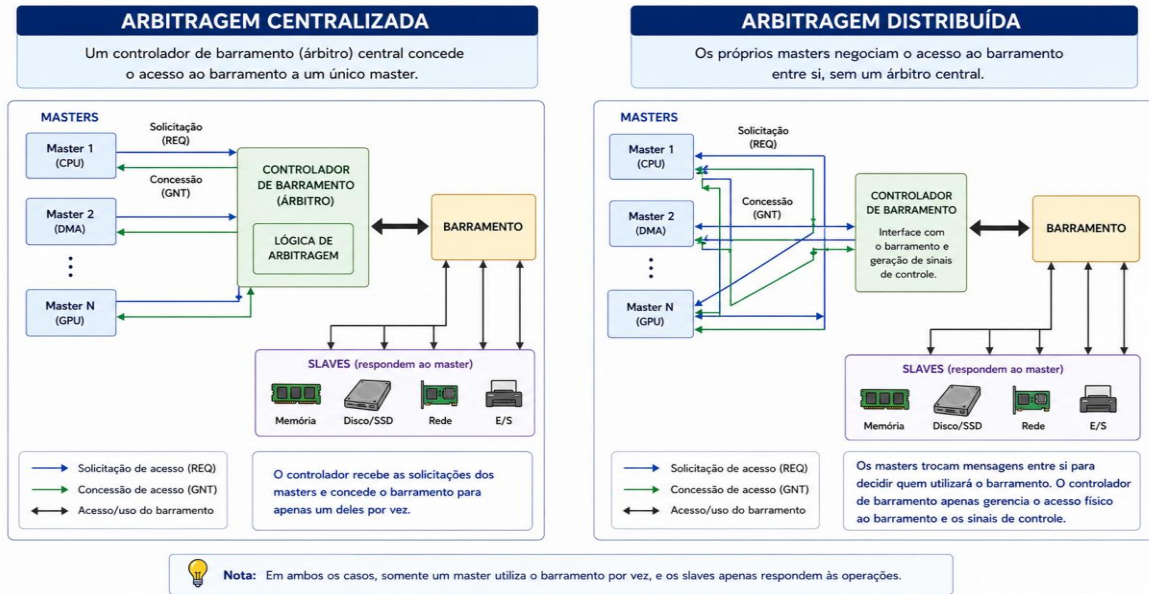


Figura 17 – Arbitragem centralizada × descentralizada — estrutura e sinais

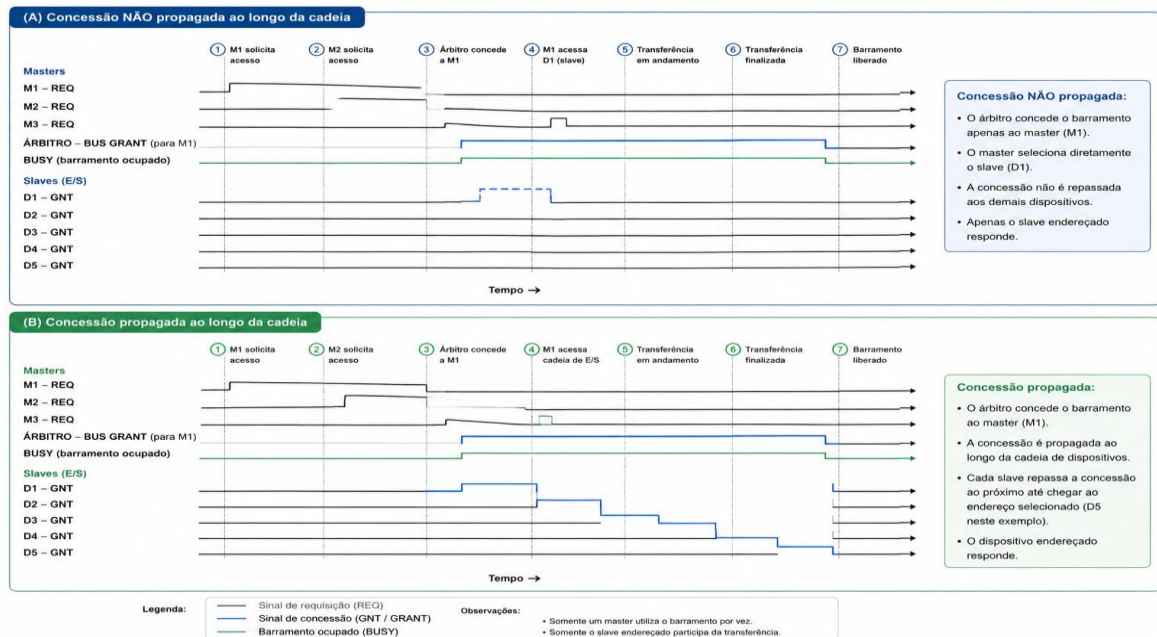


Figura 18 – Arbitragem centralizada — concessão não propagada (A) e propagada ao longo da cadeia (B)

Na arbitragem descentralizada (distribuída), não existe árbitro central: os próprios masters negociam entre si usando linhas compartilhadas de requisição, prioridade e concessão. O master com maior prioridade vence a disputa e utiliza o barramento. Isso elimina o ponto único de falha ao custo de maior complexidade por dispositivo.

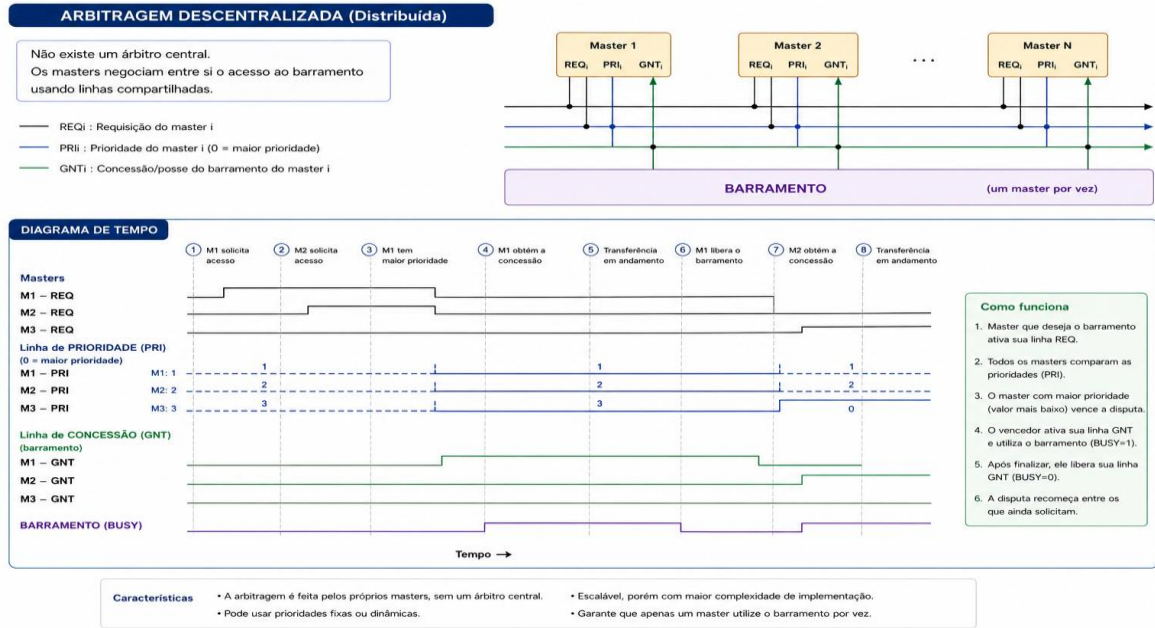


Figura 19 – Arbitragem descentralizada — diagrama de sinais e temporização

2.5.7 Temporização: síncrona e assíncrona

A temporização coordena quando cada sinal é colocado, lido e removido do barramento.

No modo **síncrono**, todas as operações são referenciadas a um clock global. A Figura 20 mostra uma leitura síncrona em sete ciclos: (1) Arbitragem/Concessão, (2) Endereço válido, (3) Comando de leitura, (4) Acesso à memória, (5) Dado válido, (6) Confirmação (ACK/READY), (7) Liberação do barramento.

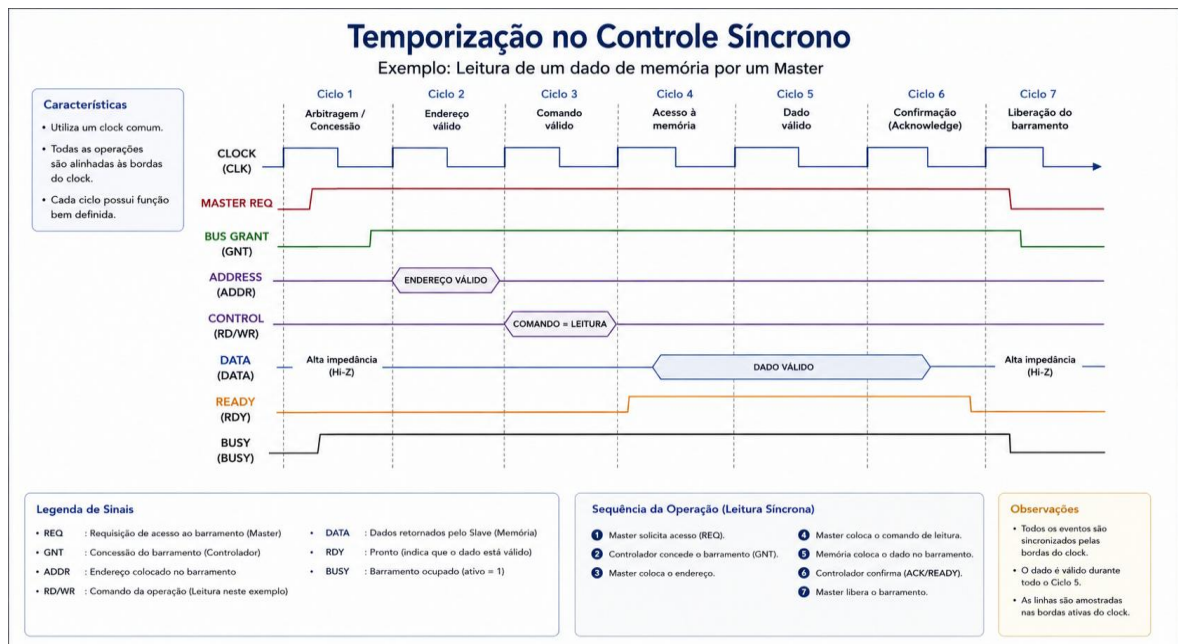


Figura 20 – Temporização síncrona — leitura de dado de memória em sete ciclos de clock

No modo **assíncrono** (*handshake*), não existe clock global: cada etapa avança apenas após o *acknowledge* do participante. A Figura 21 mostra a mesma leitura em nove etapas: REQ, GRANT, ADD + ADD ACK, RD + RD ACK, DATA + DATA ACK e liberação.

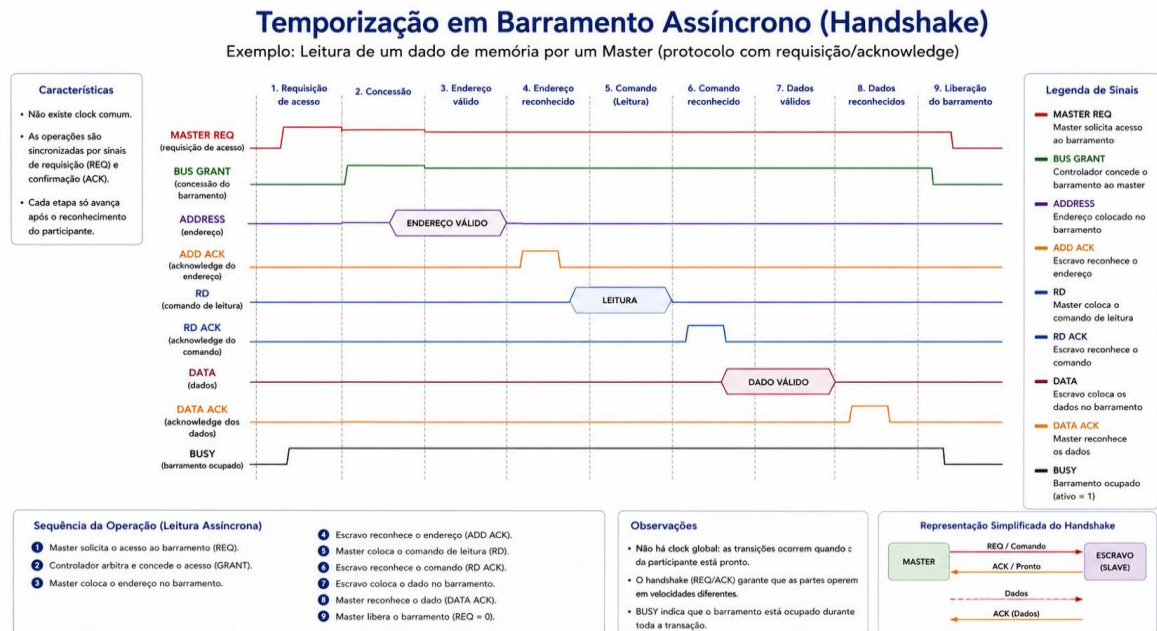


Figura 21 – Temporização assíncrona (*handshake*) — leitura de dado de memória em nove etapas confirmadas

3. PROCESSADORES

3.1 Do modelo de von Neumann ao processador moderno

A arquitetura proposta por John von Neumann nos anos 1940 estabelece o modelo conceitual mais influente da história da computação: um processador com Unidade de Controle e Unidade Lógica e Aritmética, acessando uma memória única por meio de um único barramento interno. As instruções são executadas estritamente em sequência e há apenas um núcleo de processamento.

O modelo apresenta duas limitações: (i) o *gargalo de von Neumann* — o processador fica ocioso aguardando a memória em cada ciclo; e (ii) apenas uma instrução pode estar em execução a cada instante. As duas inovações que superaram essas limitações são o *pipeline* de instruções — paralelismo temporal dentro de um único núcleo — e o *multicore* — replicação de núcleos independentes no mesmo chip.

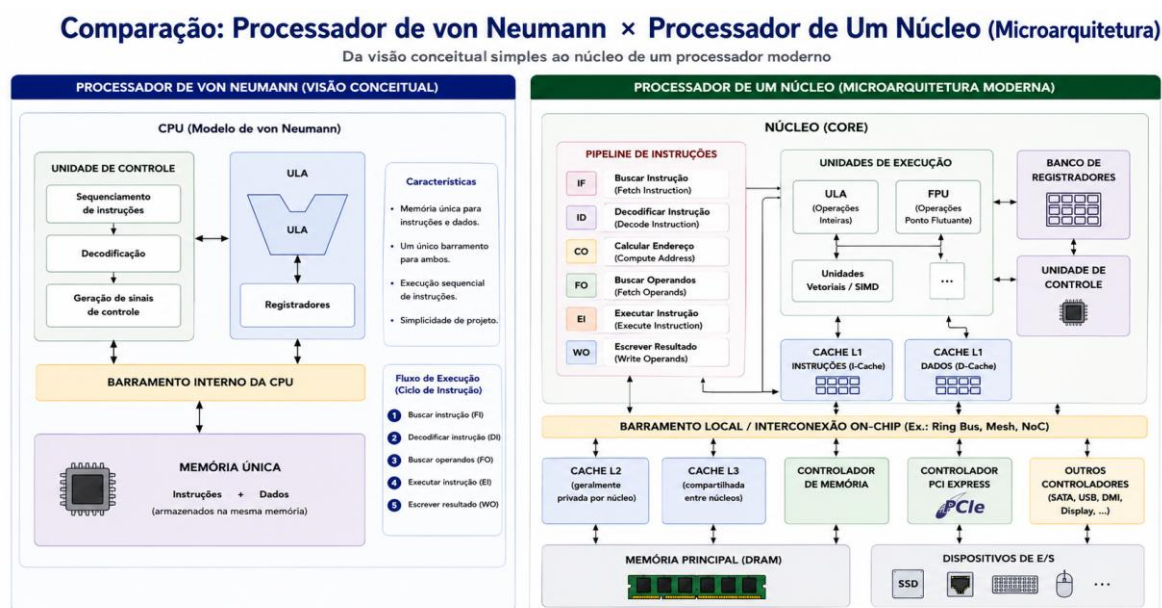


Figura 22 – Processador de von Neumann × núcleo de processador moderno com microarquitetura completa

3.2 Pipeline de instruções

3.2.1 Estágios do ciclo de instrução

O *pipeline* permite a sobreposição temporal da execução de múltiplas instruções, à semelhança de uma linha de montagem: enquanto a instrução i está no estágio EI, a instrução $i+1$ já está em FO e a instrução $i+2$ está sendo buscada. Em um pipeline de seis estágios:

- FI (*Fetch Instruction*): lê a próxima instrução apontada pelo PC.

- DI (*Decode Instruction*): decodifica o opcode e identifica o tipo de operação e os operandos.
- CO (*Compute Address*): calcula o endereço efetivo de cada operando origem.
- FO (*Fetch Operands*): lê cada operando da memória ou dos registradores.
- EI (*Execute Instruction*): realiza a operação indicada pelo opcode.
- WO (*Write Operands*): armazena o resultado final na memória ou nos registradores de destino.

>> Exemplo — estágios da instrução ADD R3, R1, R2

Considere a instrução "R3 ← R1 + R2" em uma CPU com pipeline de 6 estágios:

FI: a CPU lê os bytes da instrução ADD da memória de instruções (endereço apontado pelo PC).

DI: a UC decodifica o opcode (ADD) e identifica os operandos de origem (R1, R2) e destino (R3).

CO: como os operandos são registradores, não há cálculo de endereço de memória — estágio trivial.

FO: os valores de R1 e R2 são lidos do banco de registradores.

EI: a ULA soma R1 + R2 e produz o resultado.

WO: o resultado é escrito no registrador R3.

Enquanto esta instrução está em EI, a seguinte já está em FO e a posterior em CO — três instruções simultâneas no pipeline.

3.2.2 Características dos estágios

Para operação eficiente, o pipeline requer que: (i) os estágios tenham duração aproximadamente igual — os mais lentos criam bolhas (ciclos de espera); (ii) os estágios sejam independentes entre si — instruções sucessivas não devem depender do resultado ainda não escrito de instruções anteriores (hazards de dados); (iii) a execução em cada estágio seja simultânea e paralela; (iv) não haja conflito de acesso à memória entre estágios; e (v) haja tratamento adequado para instruções de desvio.

3.2.3 Desempenho do pipeline

O ganho de desempenho é dado por $G = Ts/pipe / Tc/pipe$. Para um pipeline de seis estágios, n instruções e tempo unitário t por estágio:

— Sem pipeline: $Ts/pipe = 6 \cdot n \cdot t$ (todas as instruções executadas em série).

— Com pipeline: $Tc/pipe = (5 + n) \cdot t$ (os primeiros 5 ciclos enchem o pipeline; a partir do 6.º, uma instrução é concluída por ciclo).

— Ganho: $G = 6n / (5 + n)$. Para n grande, G tende a 6 — o número de estágios.

A Figura 23 apresenta o diagrama de tempo para nove instruções. A partir do 6.º ciclo, todos os seis estágios estão ocupados simultaneamente — o regime pleno de operação.

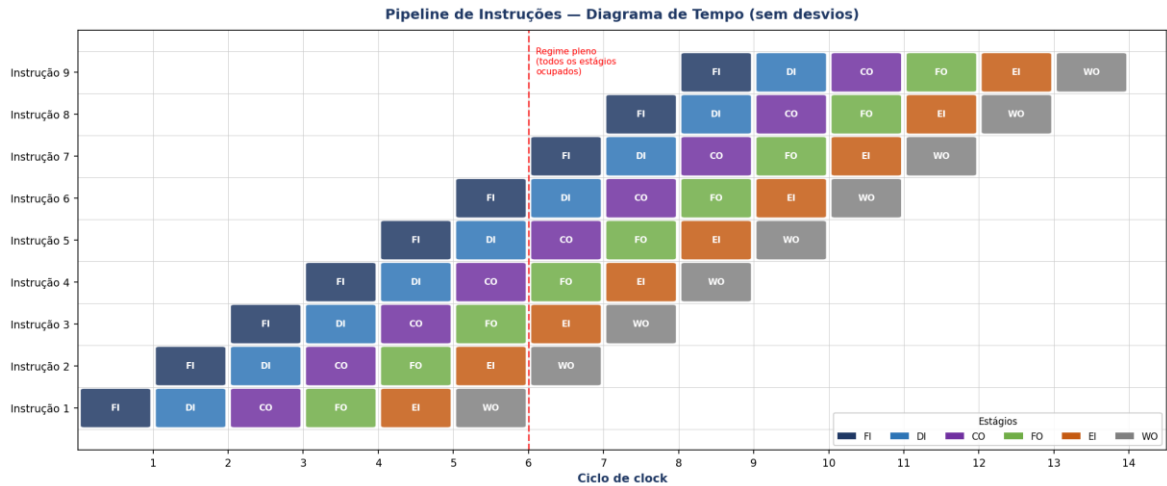


Figura 23 – Pipeline de instruções — diagrama de tempo para nove instruções; regime pleno a partir do 6.º ciclo

3.2.4 Impacto de desvios no pipeline

Instruções de desvio são a principal fonte de degradação de desempenho. Quando o processador encontra um desvio, ainda não sabe qual será a próxima instrução a executar. As instruções buscadas incorretamente após o desvio devem ser descartadas (*pipeline flush*), desperdiçando os ciclos já gastos. Em 6 estágios, um desvio pode custar até 5 ciclos desperdiçados.

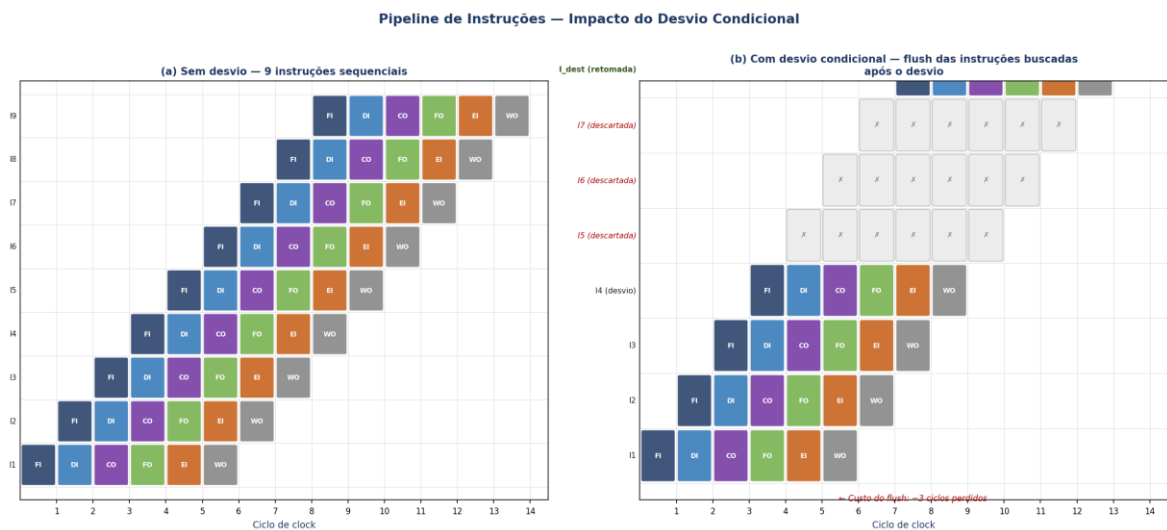


Figura 24 – Pipeline — comparação sem desvio (a) e com desvio condicional (b): custo do flush

3.3 Processadores *multicore*

Com o *pipeline*, o processador explora paralelismo temporal. O *multicore* acrescenta paralelismo espacial: múltiplos núcleos executam fluxos completamente independentes ao mesmo tempo.

Cada núcleo dispõe de todos os elementos para executar instruções autonomamente. A Figura 25 apresenta a organização interna de um processador com N núcleos.

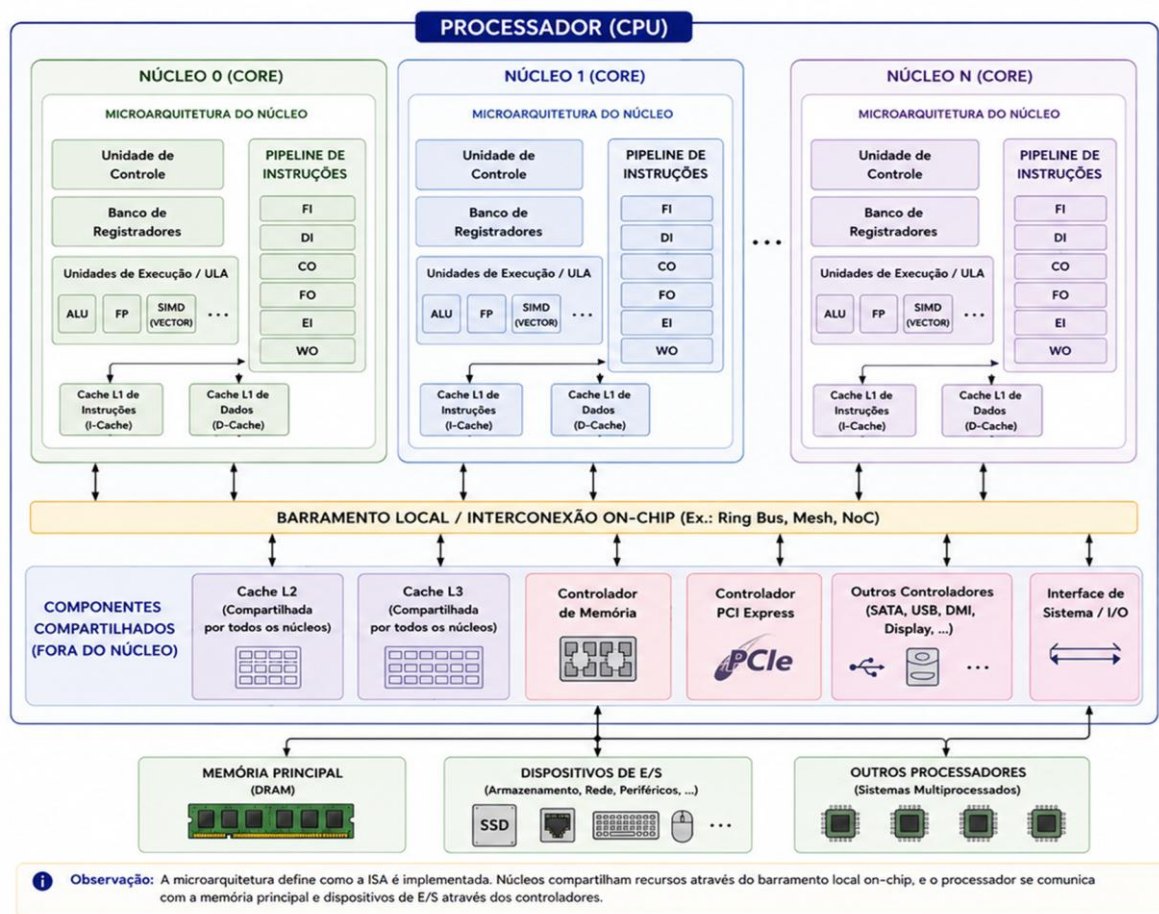


Figura 25 – Organização interna de processador multicore com N núcleos — estrutura completa

Todos os núcleos comunicam-se por um Barramento Local / Interconexão *On-Chip* (*Ring Bus*, *Mesh* ou *Network-on-Chip* — NoC) e compartilham Caches L2 e L3, Controlador de Memória, Controlador PCIe e outros controladores.

3.3.1 Compartilhamento e *overhead* de sincronização

O compartilhamento exige: (i) coerência de cache — protocolos como *MESI* (*Modified, Exclusive, Shared, Invalid*) garantem que todos os núcleos enxerguem a mesma versão do dado; (ii) sincronização — mecanismos como *mutex* e semáforo garantem exclusão mútua no acesso a dados compartilhados. Ambos introduzem *overhead* que reduz o ganho efetivo do multicore.

3.3.2 Benefícios e limitações

Os benefícios incluem: maior desempenho para cargas paralelizáveis, melhor multitarefa e maior eficiência energética — dois núcleos moderados consomem menos energia do que um único núcleo em frequência extremamente elevada, onde o fenômeno de *power wall* limita o ganho de clock.

A Lei de Amdahl estabelece que o ganho máximo é limitado pela fração sequencial do código: se 20% do programa é sequencial, o ganho máximo com infinitos núcleos é apenas 5×. *Overhead* de sincronização e contenção na memória compartilhada reduzem o ganho real abaixo do previsto.

3.4 Set de instruções

3.4.1 Conceito e ISA

O conjunto completo de instruções que o processador pode executar é chamado de set de instruções ou *ISA (Instruction Set Architecture)*. A ISA define a interface entre o software e o hardware: especifica quais operações são suportadas, como os operandos são especificados e como a memória é endereçada. Em termos de Patterson e Hennessy (2017), a ISA é "o repertório de instruções de um computador".

3.4.2 Elementos de uma instrução de máquina

Uma instrução de máquina é composta por: (i) *opcode* — código da operação; (ii) referência aos operandos de entrada; (iii) referência ao resultado — destino da operação; e (iv) referência à próxima instrução — explícita em desvios ou implícita no incremento automático do PC.

3.4.3 RISC × CISC — filosofias de projeto do set de instruções

O **CISC** (*Complex Instruction Set Computing*) concentra complexidade no hardware: instruções ricas e poderosas realizam operações complexas em um único comando, gerando código compacto. A desvantagem é que instruções de duração variável dificultam o *pipeline*. Exemplo dominante: x86/x86-64 (Intel Core, AMD Ryzen).

O **RISC** (*Reduced Instruction Set Computing*) concentra complexidade no compilador: instruções simples, de tamanho fixo e execução em um único ciclo. Conforme Patterson e Hennessy (2017), o RISC privilegia "a regularidade, a simplicidade e a boa relação custo-desempenho". Exemplos: ARM (Cortex-A, Apple M-series), RISC-V, MIPS, PowerPC.

Uma tendência importante é a convergência: processadores x86 modernos traduzem internamente suas instruções CISC em microoperações RISC-like antes de executá-las no pipeline.

Comparação RISC × CISC		
Aspecto	RISC (Reduced Instruction Set Computing)	CISC (Complex Instruction Set Computing)
Conjunto de instruções	Pequeno e simples; instruções de tamanho fixo; execução em 1 ciclo.	Grande e complexo; instruções de tamanho variável; múltiplos ciclos.
Modos de endereçamento	Poucos modos (reg-reg, load/store); operações sobre registradores.	Muitos modos (reg-mem, imediato, indireto, indexado etc.).
Registradores	Muitos registradores de propósito geral (minimiza acesso à memória).	Poucos registradores; frequente acesso à memória.
Pipeline	Altamente eficiente: instruções de duração uniforme facilitam o pipe.	Mais difícil de pipelinar: instruções de duração variável criam bolhas.
Compilador	Mais complexo: deve gerar sequências de instruções simples.	Mais simples: instruções complexas cobrem padrões de código diretamente.
Tamanho do código	Programas maiores em número de instruções (código mais verboso).	Programas menores em número de instruções (código mais compacto).
Exemplos atuais	ARM (Cortex-A, Apple M-series), RISC-V, MIPS, SPARC, PowerPC.	x86/x86-64 (Intel Core, AMD Ryzen) — domina desktops e servidores.
Tendência	Domina dispositivos móveis, IoT, servidores de cloud (AWS Graviton).	CPUs x86 incorporam técnicas RISC internamente (microoperações).

Figura 26 – Comparação RISC × CISC em oito dimensões — com exemplos de processadores atuais

3.4.4 Modos de endereçamento e tipos de instrução

Os modos de endereçamento definem como a instrução indica a localização do operando:

- **Imediato:** o valor do operando está embutido na própria instrução (ex.: *ADD R1, #5* — soma o valor 5 ao registrador R1).
- **Direto:** a instrução contém o endereço de memória do operando (ex.: *LOAD R1, 1000* — carrega o conteúdo da posição 1000 em R1).
- **Indireto:** a instrução aponta para uma posição que contém o endereço do operando — dois acessos à memória são necessários.
- **Relativo (indexado ou baseado):** o endereço efetivo é calculado somando um registrador base a um deslocamento fixo (ex.: *LOAD R1, 8(R2)* — endereço = R2 + 8).

Os tipos de ação classificam-se em: aritméticas e lógicas (*ADD, SUB, AND, OR*); desvio condicional (*BEQ, BNE*) e incondicional (*JUMP, CALL, RET*); transferência de dados (*LOAD, STORE, MOV*); e controle do processador (*NOP, HALT*).

3.4.5 Número de operandos e ciclo de instrução

Quanto ao número de campos de operando explícitos, as instruções classificam-se em: zero-op (sem operandos explícitos — operam sobre registradores implícitos), monádicas (um

operando — ex.: NOT, NEG), diádicas (dois operandos — ex.: ADD R1, R2) e triádicas (três operandos — ex.: ADD R3, R1, R2, comuns em arquiteturas RISC como ARM e RISC-V).

O ciclo de instrução descreve o fluxo completo de execução, desde a busca até o armazenamento do resultado. A Figura 27 apresenta o fluxograma do ciclo, com caminhos alternativos para instruções com e sem operandos em memória, tratamento de desvios e atualização do contador de programa.

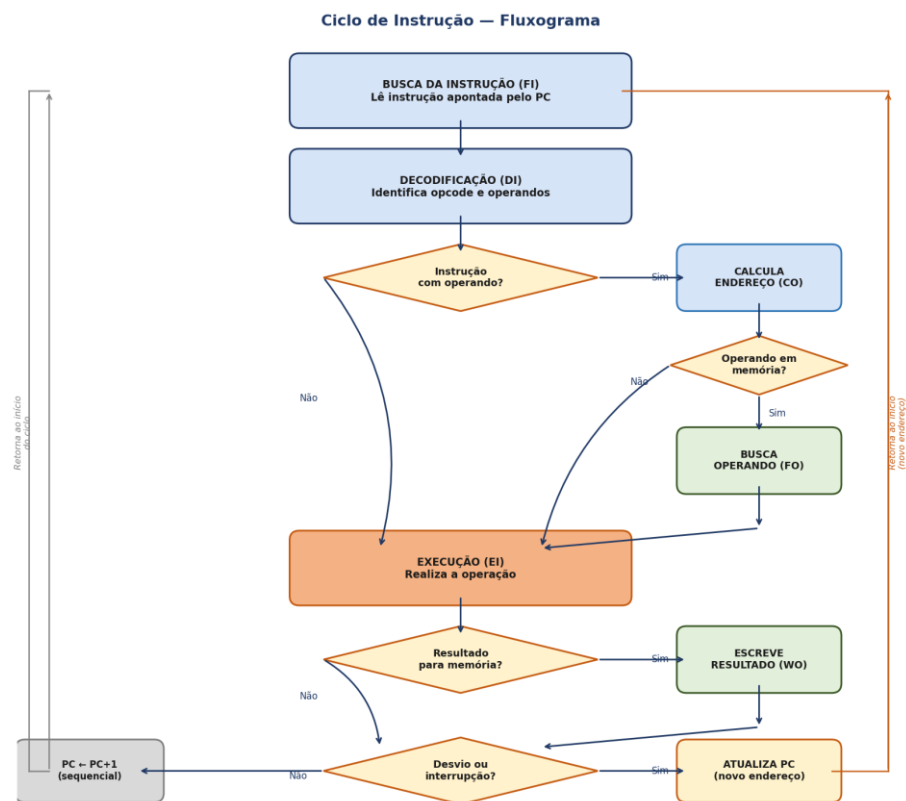


Figura 27 – Ciclo de instrução — fluxograma completo com caminhos para operandos em memória, desvio e retorno ao início

REFERÊNCIAS

PATTERSON, David A.; HENNESSY, John L. Organização e projeto de computadores: a interface hardware/software. 5. ed. Rio de Janeiro: Elsevier, 2017.

PCI-SIG. PCI Express Base Specification Revision 5.0. Beaverton, 2019. Disponível em: <https://pcisig.com>. Acesso em: jun. 2026.

STALLINGS, William. Arquitetura e organização de computadores. 10. ed. São Paulo: Pearson, 2017.

TANENBAUM, Andrew S.; AUSTIN, Todd. Organização estruturada de computadores. 6. ed. São Paulo: Pearson, 2013.